

INFORME DE MEJORAS Y LIMPIEZA DEL PROYECTO PERFUM LAB

Este documento resume las mejoras realizadas a mi proyecto Perfum Lab tomando como base la evaluacion de UI/UX y calidad de software realizada por un compa?ero de clase.

Las recomendaciones principales fueron mejorar validaciones, agregar un modulo de clientes con correo electronico, agregar funciones de Excel y ordenar la interfaz sin cambiar completamente el dise?o.

Despues de implementar esas mejoras, tambien se limpio el proyecto para dejarlo listo para subir al repositorio, usando solamente JSON como almacenamiento de datos.

1. Analisis inicial

Antes de hacer cambios se reviso la estructura del proyecto.

Perfum Lab es una aplicacion de escritorio hecha en Python con Tkinter y ttk.

El codigo esta organizado por vistas, controladores, modelos, base de datos, ventas, facturas y reportes.

La base activa del sistema trabaja con archivos JSON dentro de database/json.

Por eso se mantuvo esa arquitectura y se eliminaron los restos de SQLite que ya no se necesitaban.

2. Validaciones implementadas

Se reforzaron las validaciones para evitar guardar informacion incompleta o incorrecta.

Ahora el SKU y el nombre del producto son obligatorios.

No se permite registrar un SKU duplicado.

El precio debe ser un numero valido y mayor que 0.

El costo, el stock actual y el stock minimo no pueden ser negativos.

Las entradas, salidas y ajustes de inventario validan cantidades correctas.

Si no hay suficiente stock, el sistema muestra un mensaje claro.

Tambien se mejoraron los mensajes para que el usuario sepa exactamente que debe corregir.

3. Modulo de clientes

Se agrego un nuevo apartado llamado Clientes en el menu principal.

Este modulo permite registrar, buscar, editar, desactivar y listar clientes.

Los datos manejados son ID, nombre, correo electronico, telefono, direccion y estado.

El correo se valida para aceptar formatos correctos como cliente@gmail.com o usuario@hotmail.com.

No se permite registrar dos clientes con el mismo correo electronico.

Los archivos creados fueron app/models/cliente.py, app/controllers/clientes_controller.py y app/views/clientes_view.py.

4. Funciones de Excel

Se agregaron botones de Exportar Excel e Importar Excel dentro de Productos e inventario.

La exportacion genera un archivo .xlsx con ID, SKU, nombre, marca, categoria, costo, precio, stock actual, stock minimo y estado.

La importacion permite seleccionar un archivo .xlsx y registrar productos nuevos.

Antes de guardar se validan campos obligatorios, numeros y SKU duplicados.

Si una fila del Excel tiene un problema, se muestra el numero de fila y no se guarda el lote incompleto.

La dependencia usada para Excel es openpyxl.

5. Mejoras de interfaz

No se cambio completamente el dise?o del programa.

Se mantuvo el estilo visual existente definido en app/ui_theme.py.

La pantalla de Clientes sigue el mismo orden visual del sistema: barra de busqueda, tabla y formulario.

En Productos se organizo el grupo Archivos para las opciones de Excel.

Los mensajes de error y confirmacion ahora usan titulos mas claros como Productos, Inventario, Excel o Clientes.

6. Limpieza de base de datos

Como el sistema ahora usa JSON, se limpio database/json para que todos los archivos queden vacios y listos para empezar desde cero.

Los archivos categorias.json, clientes.json, productos.json, movimientos_inventario.json, ventas.json, detalle_venta.json, facturas.json y usuarios.json quedaron como listas vacias.

Esto evita subir datos de prueba, clientes, productos o ventas personales al repositorio.

Tambien se simplifico app/database/json_storage.py para que los datos iniciales sean vacios.

Si los archivos JSON no existen, el sistema los puede crear nuevamente vacios con crear_db.py.

7. Eliminacion de SQLite y carpetas generadas

Se elimino el archivo de base de datos SQLite local porque ya no se usa en el sistema.

Tambien se elimino el archivo de esquema SQL que ya no era necesario para el funcionamiento actual.

Se limpio la carpeta de respaldos de la migracion anterior.

Se eliminaron carpetas generadas por PyInstaller y Python, como build, dist, build_actualizado, dist_actualizado, .uv-cache y __pycache__.

Estas carpetas se pueden volver a generar cuando se necesite crear un nuevo ejecutable.

8. Archivos principales modificados o creados

README.md se actualizo con instrucciones claras para instalar, ejecutar y generar el .exe.

requirements.txt incluye openpyxl y pyinstaller.

PerfumLab.spec ahora empaqueta los JSON limpios y no archivos de SQLite.

app/database/json_storage.py ahora inicializa datos vacios.

database/json contiene archivos vacios listos para empezar.

tests/test_qa_core.py crea sus propios datos temporales para pruebas, sin depender de la

base real.

9. Verificacion realizada

Se verifico que los archivos JSON del proyecto quedaron vacios.

Se reviso que no quedaran carpetas generadas innecesarias en la raiz del proyecto.

Se reviso el estado de Git para identificar los archivos modificados, eliminados y nuevos antes del push.

La idea es subir solo codigo, documentacion util y archivos JSON limpios.

10. Conclusion

Con estos cambios, Perfum Lab queda mas ordenado para trabajar en equipo.

El proyecto conserva su arquitectura, pero ahora tiene mejores validaciones, modulo de clientes, soporte de Excel y una base JSON limpia.

Tambien queda mas claro como generar el ejecutable en otra computadora siguiendo el README.